



Cross-implementation re-verification

pid-rs estimators, lattices and mathematical results: a second implementation, exact tests, numerical and verification corrections

Sepehr Mahmoudian

26 September 2026

Cite this report as: Sepehr Mahmoudian (2026). *Cross-implementation re-verification of pid-rs estimators and mathematical results*. pid-rs repository report. Include the exact repository commit or release that you used. The software citation is in CITATION.cff. Cite the defining method papers separately; Section 15 lists them.

Summary

This report records a second implementation of the main pid-rs estimators, written from the published definitions, and compares it with pid-rs. It also re-derives several repository results, tests one repository bound in exact arithmetic, and records one defect and one citation error that the review found. The checks ran on commit d1f401a and again on commit 0e96b2b, which contains the two fixes below. Section 14.7 records later defects found in two evidence checkers; those defects do not change the retained estimator outputs.

The main results are:

1. **Categorical shared exclusions and $I_{\min}$.** The second implementation reproduced every averaged informative, misinformative and net atom of the Makkeh–Gutknecht–Wibral (MGW) shared-exclusions decomposition for 2, 3 and 4 sources. It also reproduced every Williams–Beer $I_{\min}$ redundancy and atom for 2 and 3 sources. The largest absolute difference over 60 seeded systems was 7.7×10^{-16} nats. An existing exact-rational oracle test already covers four fixed four-source tables; this check adds 20 randomized four-source systems.
2. **Continuous estimators.** A brute-force implementation reproduced the KSG1 mutual information, the local terms of $\hat{I}(S_1; T)$, the Ehrlich source-disjunction redundancy, the PID2 atoms and every computed PID3 lattice redundancy. The largest absolute difference over six seeded systems with Gaussian inputs was 8.9×10^{-16} nats. The KSG comparison covers both neighbour paths of pid-rs: brute force and the exact Chebyshev kd-tree.
3. **Test suite.** The complete pid-core test suite passed in release mode with all features: 725 tests on d1f401a and 728 on 0e96b2b, with 0 failures and 6 ignored items each time.
4. **A repository bound in exact arithmetic.** An exact-rational test of the one- Λ local bound in the [dependency-colored concentration document](#) found no violation in 634,112 evaluations. The informative and misinformative components attain the bound exactly. An earlier binary64 version of the test reported ratios up to 4. Section 6 shows that its construction broke the premise that both laws have total mass one.
5. **One defect, now fixed.** The raw percentile summaries of the live resampling routines computed their lower and upper indices with two separate binary64 expressions. For 22 of the 99 two-decimal values of α , some replicate count $B \leq 20000$ dropped one more value from one tail than from the other. The indices now use one tail count for both ends (Section 7).





6. **One citation correction.** `pid-rs` attributed its $k - 2$ intrinsic-dimension normalization to MacKay and Ghahramani (2005). The primary source is Levina and Bickel (2004), Section 3, below their Equation (8). The estimator and its outputs do not change (Section 9).
7. **Formal packages.** A semantic audit of the 14 Lean directories and the two SMT packages found no mathematical or formal defect. Every checked statement matches its prose, no checked premise set is contradictory, and every SMT background is satisfiable. One publication record named stale file hashes; it is corrected (Section 14).

Apart from the percentile defect, no defect was found in any checked formula, lattice, estimator implementation, or proof step. This result is bounded. It does not establish estimator calibration, consistency, population validity, or application value. Section 1 states the scope, the evidence classes and the limits of independence.

1. Scope, objects and evidence

1.1 Checked objects

Object	Defining source	pid-rs route	Evidence in this report	Boundary
MGW categorical shared exclusions	Makkeh, Gutknecht and Wibral, arXiv:2002.03356v5	<code>discrete_sxpid2</code> , <code>discrete_sxpid3</code> , <code>discrete_sxpid_n</code>	Second implementation on 60 systems; re-derivation	Empirical plug-in law only
Williams–Beer $I_{\min}$	Williams and Beer, arXiv:1004.2515v1	<code>imin_pid2</code> , <code>imin_pid3</code>	Second implementation on 40 systems	A different redundancy measure
KSG1 mutual information	Kraskov, Stögbauer and Grassberger (2004), DOI 10.1103/PhysRevE.69.066138	<code>ksg_mi</code> , <code>ksg_local_mi_te rms</code> , <code>ksg_mi_concat_xy</code>	Second implementation; two lemmas	Implementation agreement only
Ehrlich source-disjunction redundancy	Ehrlich et al., arXiv:2311.06373v3	<code>isx_redundancy</code>	Second implementation	Restricted continuous domain
PID2 and PID3 compositions	Ehrlich et al. atom construction	<code>pid2_isx</code> , <code>incomplete_pid3_diagnostic</code> , <code>research_pid3_isx</code>	Second implementation	Composition, not calibration
Raw resampling percentiles	Project-defined summaries	<code>block_bootstrap</code> , <code>block_bootstrap_paired</code> , <code>bootstrap_rows_stats</code> , <code>bootstrap_quantized_sxpid2</code>	Exact index analysis; defect fixed with tests	No coverage claim
Levina–Bickel intrinsic dimension	Levina and Bickel (2004)	<code>intrinsic_dimension_report</code>	Re-derivation; source check	Diagnostic only
Repository bounds and envelopes	Repository documents listed in Section 8	Documentation	Re-derivation; exact stress test	No new theorem
Formal packages	Package documents under <code>audit/formal</code>	14 Lean directories and two SMT packages	Statement-to-prose audit; SMT background test	No proof replay

Section 11 gives the module paths and feature gates of these functions.





1.2 Evidence classes and independence

This report uses three evidence classes of the repository's **evidence firewall**:

- **Execution evidence.** A test or checker bounded to exact inputs, toolchain and assumptions. It is not a universal theorem or a general validation result.
- **Model review.** An AI-assisted session read the code, wrote the second implementation and re-derived the formulas. Two further sessions reviewed a draft of this report and one reviewed the revised draft; the author session performed the language and layout lenses and the audit of the formal packages in Section 14. This is advisory. It is not human, institutional or independent review.
- **Documentation.** Source quotations, retrieval records and the correction record.

No formal proof and no human line review are claimed.

The repository records five independence dimensions separately:

Dimension	Status	Reason
Semantic	Partial	One reviewer read the papers and the pid-rs code. A shared misreading of a definition would pass both implementations. The worked example in Section 2.3 matches the published MGW value.
Implementation	Partial	Different language and algorithms: exact rational probabilities, direct antichain enumeration, a dense linear solve instead of ordered subtraction, and dense distance matrices instead of a kd-tree. The same reviewer had read the Rust code.
Custody	None	One machine, one session and one repository checkout.
Institutional	None	No separate organization took part.
Data	None	The Python checks read rows written by the Rust generators and compare them with pid-rs outputs from the same files. No external data set is used.

This is retrospective, exploratory evidence. The checkers fail when a difference exceeds 10^{-12} nats, but that tolerance was chosen after the results were known. It is a regression guard, not a sealed acceptance rule.

1.3 Correspondence edges

The repository separates five correspondence edges. This report touches them as follows:

1. **Source to repository specification.** The formulas of Sections 2 and 4 were re-derived from the primary sources, and the Levina–Bickel locator was corrected. Model review only.
2. **Repository specification to formal model.** Section 14 reviews selected statements and definitions against their prose. It does not establish every correspondence edge or replay proofs.
3. **Formal model to executable algorithm.** Not examined.
4. **Executable algorithm to numeric execution.** The second implementation agrees to rounding on the stated inputs (execution evidence). Sections 4 and 7 add lemmas about the binary64 behavior of the neighbour search and the percentile indices.
5. **Implementation output to scientific estimand and application.** Not established. No calibration, sampling or application claim is made.





2. Notation

All logarithms are natural logarithms. Information is in nats.

- $S_1, \dots, S_m$ are the sources and T is the target. Each variable takes values in a finite alphabet. A complete realization is $z = (s_1, \dots, s_m, t)$.
- p is a probability law on the finite product alphabet $\mathcal{Z}$. The empirical law of n rows is $\hat{p}_n(z) = N(z)/n$, where $N(z)$ counts the rows equal to z .
- For a nonempty set $a \subseteq \{1, \dots, m\}$, s_a is the restriction of s to a . For a realization $y \in \mathcal{Z}$, y_a is the restriction of its sources to a , and y_T is its target value.
- An **antichain** α is a nonempty family of nonempty source sets such that no member contains another member. There are 4, 18 and 166 antichains for $m = 2, 3, 4$.
- The **redundancy order** is $\alpha \preceq \beta$ if every $b \in \beta$ contains some $a \in \alpha$.

2.1 Shared-exclusion events and pointwise terms

For an antichain α and a realization $z = (s, t)$, MGW use two events:

$$\mathbf{a}_\alpha(z) = \{y \in \mathcal{Z} : y_a = s_a \text{ for at least one } a \in \alpha\}, \quad \mathbf{t}(z) = \{y \in \mathcal{Z} : y_T = t\}.$$

The event $\mathbf{a}_\alpha(z)$ is an OR across the collections of α . Inside one collection it is an AND: all sources in a must match. If $p(z) > 0$, then z belongs to both events, so both events and their intersection have positive probability. The pointwise informative, misinformative and net terms are:

$$i_\alpha^+(z) = -\log p(\mathbf{a}_\alpha(z)), \quad i_\alpha^-(z) = \log \frac{p(\mathbf{t}(z))}{p(\mathbf{t}(z) \cap \mathbf{a}_\alpha(z))},$$

$$i_\alpha^{\text{sx}}(z) = i_\alpha^+(z) - i_\alpha^-(z) = \log \frac{p(\mathbf{t}(z) \cap \mathbf{a}_\alpha(z))}{p(\mathbf{t}(z)) p(\mathbf{a}_\alpha(z))}.$$

2.2 From cumulative terms to atoms

Let $Z_{\alpha\beta} = 1$ if $\beta \preceq \alpha$ and $Z_{\alpha\beta} = 0$ otherwise. The matrix Z is invertible because it is unitriangular in any topological order of the lattice. Write $M = Z^{-1}$. For each component $u \in \{+, -, \text{sx}\}$, the pointwise atoms are

$$\pi_\alpha^u(z) = \sum_\beta M_{\alpha\beta} i_\beta^u(z),$$

and the averaged atoms are $\Pi_\alpha^u = \sum_z p(z) \pi_\alpha^u(z)$. Equivalently, every cumulative term is the sum of the atoms at or below its node:

$$i_\alpha^u(z) = \sum_{\beta \preceq \alpha} \pi_\beta^u(z).$$

For a singleton node $\alpha = \{\{r\}\}$, the net term is $\log(p(s_r, t)/(p(s_r)p(t)))$, so its average under p is $I(S_r; T)$. At the top node $\{\{1, \dots, m\}\}$ the average is $I(S_1, \dots, S_m; T)$. These two facts give the unique and synergy formulas below.





2.3 Worked example: two-bit XOR

Let S_1 and S_2 be independent uniform bits and let $T = S_1 \oplus S_2$. Take $z = (0, 0, 0)$ and the redundancy node $\alpha = \{\{1\}, \{2\}\}$.

1. $\mathbf{a}_\alpha(z) = \{y : y_1 = 0 \text{ or } y_2 = 0\}$ contains three of the four equally likely source pairs. Hence $p(\mathbf{a}_\alpha(z)) = 3/4$.
2. $\mathbf{t}(z) = \{y : y_T = 0\}$ contains $(0, 0, 0)$ and $(1, 1, 0)$. Hence $p(\mathbf{t}(z)) = 1/2$.
3. Only $(0, 0, 0)$ lies in both events. Hence $p(\mathbf{t}(z) \cap \mathbf{a}_\alpha(z)) = 1/4$.
4. Therefore $i_\alpha^{\text{sx}}(z) = \log \frac{1/4}{(1/2)(3/4)} = \log \frac{2}{3}$.

Every realization gives the same value by symmetry. So the averaged redundancy is $\log(2/3) \approx -0.405$ nats. Each single source has $I(S_i; T) = 0$, so each unique atom is $0 - \log(2/3) = \log(3/2)$. The synergy atom is $\log 2 - 0 - 0 + \log(2/3) = \log(4/3)$. The four atoms sum to $\log(\frac{2}{3} \cdot \frac{9}{4} \cdot \frac{4}{3}) = \log 2 = I(S_1, S_2; T)$.

2.4 Williams–Beer redundancy

For a source set a and a target value t with $p(t) > 0$, the specific information is

$$I_{\text{spec}}(S_a; t) = \sum_{s_a : p(s_a, t) > 0} p(s_a | t) \log \frac{p(t | s_a)}{p(t)}.$$

The Williams–Beer redundancy of an antichain is $I_{\min}(\alpha) = \sum_t p(t) \min_{a \in \alpha} I_{\text{spec}}(S_a; t)$. Its atoms follow from the same Möbius inversion. It is a different measure from shared exclusions; the two must not be pooled.

3. Categorical checks

3.1 Method

The generator `rust/audit_dump_discrete.rs` draws 60 systems from a fixed seed. The number of sources cycles through 2, 3 and 4, so each count has 20 systems. Each source alphabet has 2 or 3 states, the target has 2 or 3 states, and each system has between 8 and 127 rows. Four target rules alternate. Rule 1 draws an independent random target. Rule 2 takes the sum of all sources modulo the target alphabet size, which is the parity for a binary target. Rule 3 takes $s_1 + s_2$ modulo the target alphabet size, but replaces each row's target by a random value with probability 1/4. Rule 4 takes $s_1 s_2 + s_m$ modulo the target alphabet size. Small row counts leave some cells empty, so the empirical laws are not uniform and often lack full support.

The checker `python/check_discrete.py` recomputes every quantity from the rows. It does not call `pid-rs`:

1. It forms the empirical law with exact rational probabilities.
2. It enumerates antichains as families of Python sets and tests the antichain condition directly.
3. For each observed realization, it evaluates the informative and misinformative pointwise terms from the event definitions in Section 2.1. The net term is their difference. It also computes the mutual information of every nonempty source subset.
4. It solves $Zx = c$ with a dense linear solver. `pid-rs` instead subtracts lower atoms in a topological order.
5. It averages the atoms under the empirical law and compares every component.





The checker requires exactly 60 systems, 20 for each source count, and the expected length of every output list. It rejects duplicate JSON keys, duplicate antichains, non-numeric values and antichain sets that differ from its own enumeration. A retained self-test, `python/checker_self_test.py`, confirms that 17 malformed inputs make the two checkers fail.

3.2 Results

The first table gives the largest absolute difference in nats for the shared-exclusions atoms. “General path” is `discrete_sxpid_n`; “specialized path” is `discrete_sxpid2` or `discrete_sxpid3`. The specialized comparison checked the informative and misinformative components; their difference is the net atom.

Sources and path	Informative	Misinformative	Net
2, general	1.1×10^{-16}	1.1×10^{-16}	1.7×10^{-16}
2, specialized	1.1×10^{-16}	1.1×10^{-16}	not compared
3, general	1.4×10^{-16}	1.7×10^{-16}	2.2×10^{-16}
3, specialized	1.4×10^{-16}	1.7×10^{-16}	not compared
4, general (166 antichains)	4.9×10^{-16}	6.0×10^{-16}	7.7×10^{-16}

The second table covers the remaining quantities.

Quantity	Largest absolute difference (nats)
Joint mutual information, all source counts	0
Mutual information of every source subset, general path	0
Two-source MI terms, specialized path	0
Sum of net atoms minus joint mutual information	2.2×10^{-16}
Two-source $I_{\min}$ redundancy, atoms and MI terms	5.6×10^{-17}
Three-source $I_{\min}$ atoms	1.4×10^{-16}
Three-source $I_{\min}$ redundancies	2.2×10^{-16}

The full output is `results/check_discrete.txt`.

3.3 Relation to existing tests, interpretation and limits

The repository already had a from-definition comparison for four sources. The test `every_realizable_table_matches_public_sxpid_and_every_bound_holds` in `support_change_tolerant_sxpid_oracle.rs`, added in commit `e57b34e` on 24 July 2026, compares the averaged and pointwise atoms of `discrete_sxpid_n` with an exact-rational and high-precision Decimal oracle. Its fixed tables include two, three and four sources. The present check adds 20 randomized four-source systems with other alphabets, row counts and target rules, and a second algorithm for the Möbius inversion.

All differences are a few units of binary64 rounding. This is implementation agreement with the published definitions on these 60 laws. It does not show that plug-in atoms estimate population atoms well, and it does not test alphabets or row counts outside the stated ranges.





4. Continuous checks

4.1 KSG1 and the count range

For rows $i = 1, \dots, N$, let $d_X(i, j)$ and $d_Y(i, j)$ be Chebyshev distances and let $d(i, j) = \max\{d_X(i, j), d_Y(i, j)\}$. Let ε_i be the k -th smallest value of $d(i, j)$ over $j \neq i$. The strict marginal counts are $n_x(i) = \#\{j \neq i : d_X(i, j) < \varepsilon_i\}$ and the same for $n_y(i)$. KSG1 estimates

$$\hat{I} = \psi(k) + \psi(N) - \frac{1}{N} \sum_{i=1}^N [\psi(n_x(i) + 1) + \psi(n_y(i) + 1)],$$

where ψ is the digamma function. For a positive integer r , $\psi(r) = H_{r-1} - \gamma$, with $H_0 = 0$ and $H_r = \sum_{j=1}^r 1/j$. The Euler constant cancels:

$$\psi(k) + \psi(N) - \psi(a) - \psi(b) = H_{k-1} + H_{N-1} - H_{a-1} - H_{b-1}.$$

pid-rs evaluates this harmonic form from one prefix table.

Lemma 1 (count range). Assume that exactly $k - 1$ other rows satisfy $d(i, j) < \varepsilon_i$ and exactly one row satisfies $d(i, j) = \varepsilon_i$. Then $k - 1 \leq n_x(i) \leq N - 1$.

Proof. Each of the $k - 1$ interior rows satisfies $d_X(i, j) \leq d(i, j) < \varepsilon_i$, so it is counted in $n_x(i)$. There are only $N - 1$ other rows. $\square$

If the k -th neighbour shell of any row does not have exactly $k - 1$ interior points and one boundary point, pid-rs fails the whole call with `AmbiguousKthNeighborShell`. So on every returned estimate the harmonic arguments satisfy $k \leq n_x(i) + 1 \leq N$, which is the index range that the prefix table requires. The same argument gives $k \leq n_\alpha(i) \leq N$ and $k \leq n_T(i) \leq N$ for the self-inclusive counts of Section 4.3.

4.2 The kd-tree prunes exactly under rounding

Lemma 2. For a query q and an axis-aligned box with corners $\ell \leq h$, the tree uses the bound

$$g = \max_c \max\{\text{fl}(\ell_c - q_c), \text{fl}(q_c - h_c), 0\},$$

where fl is binary64 rounding to nearest. For every point x in the box, g is at most the computed Chebyshev distance $\max_c \text{fl}(|q_c - x_c|)$.

Proof. Fix a coordinate c . If $q_c < \ell_c \leq x_c$, then $x_c - q_c \geq \ell_c - q_c \geq 0$ exactly. Rounding to nearest is monotone, and $\text{fl}(q_c - x_c) = -\text{fl}(x_c - q_c)$. Hence $\text{fl}(|q_c - x_c|) = \text{fl}(x_c - q_c) \geq \text{fl}(\ell_c - q_c)$. The case $x_c \leq h_c < q_c$ is symmetric. In every other case the coordinate contributes a nonpositive gap, and the bound uses 0. Take the maximum over c . $\square$

Corollary. The tree evaluates leaf distances with the same scalar fold as the brute-force Chebyshev distance, and it prunes a box only when $g > r$ for the query radius r . By Lemma 2, a pruned box contains no point whose computed distance is at most r . So both paths count the same points strictly inside and exactly on the radius, and they return the same counts. The repository's parity tests check this on data.





4.3 Ehrlich source-disjunction redundancy

For two sources of equal dimension, the redundancy estimator uses the joint distance

$$d_{\cup}(i, j) = \max\{d_T(i, j), \min\{d_{S_1}(i, j), d_{S_2}(i, j)\}\}.$$

Here ε_i is the k -th smallest value of $d_{\cup}(i, j)$ over $j \neq i$. The counts include the query row itself:

$$n_{\alpha}(i) = 1 + \#\{j \neq i : \min\{d_{S_1}, d_{S_2}\}(i, j) < \varepsilon_i\}, \quad n_T(i) = 1 + \#\{j \neq i : d_T(i, j) < \varepsilon_i\}.$$

The local term is $\psi(k) + \psi(N) - \psi(n_{\alpha}(i)) - \psi(n_T(i))$, and the estimate is its mean. The minimum of the two source distances describes the union of the two source balls. This is the continuous analogue of the OR event in Section 2.1. For three sources and a general antichain α , pid-rs uses the source distance $\min_{a \in \alpha} \max_{r \in a} d_{S_r}$. The inner maximum is the Chebyshev distance of the concatenated collection. This estimator has only a brute-force path.

Let $\hat{R}$ be the redundancy estimate and let $\hat{I}$ denote KSG1 estimates. The PID2 atoms are

$$\widehat{\text{Red}} = \hat{R}, \quad \widehat{\text{Unq}}_r = \hat{I}(S_r; T) - \hat{R} \quad (r = 1, 2), \quad \widehat{\text{Syn}} = \hat{I}(S_1, S_2; T) - \hat{I}(S_1; T) - \hat{I}(S_2; T) + \hat{R}.$$

4.4 Method and results

The generator `rust/audit_dump_continuous.rs` draws six systems with (N, d, k) equal to $(60, 1, 1)$, $(60, 1, 3)$, $(150, 2, 3)$, $(300, 1, 5)$, $(400, 2, 4)$ and $(200, 1, 2)$. Per coordinate, $X \sim \mathcal{N}(0, 1)$, $Y = 0.5X + \mathcal{N}(0, 1)$ and $Z \sim \mathcal{N}(0, 1)$. The target is $X + 0.7Y + XZ + 0.5W$ in systems 1, 3 and 5 and $X + 0.7Y + 0.3Z + 0.5W$ in systems 2, 4 and 6, with independent standard normal W . The inputs are Gaussian; the target of systems 1, 3 and 5 is not, because of the product XZ . The sources are X , Y and Z . Systems with $N \geq 128$ exercise the kd-tree path of the KSG terms.

The checker `python/check_continuous.py` builds dense Chebyshev distance matrices and evaluates the formulas above. It computes digamma values at integers from $\psi(r) = H_{r-1} - \gamma$, with H summed by `math.fsum` over binary64 reciprocals and γ rounded to binary64. These values are accurate to rounding, not exact. It compares local terms for $\hat{I}(S_1; T)$ only. It fails if the research full PID3 lattice is absent.

Quantity	Largest absolute difference (nats)
KSG1 estimates $\hat{I}(S_1; T)$, $\hat{I}(S_2; T)$, $\hat{I}(S_1, S_2; T)$	5.6×10^{-16}
KSG1 local terms of $\hat{I}(S_1; T)$	8.9×10^{-16}
Ehrlich redundancy	5.0×10^{-16}
PID2 atoms	5.0×10^{-16}
Incomplete and full PID3 lattice redundancies	6.7×10^{-16}

The incomplete PID3 diagnostic abstained on 18 lattice entries in total, as designed; the checker counts these abstentions and compares every produced value. The full output is `results/check_continuous.txt`. The agreement is implementation agreement only. These data satisfy the absolute-continuity premise, but the check says nothing about estimator bias, variance or calibration.





5. Complete test suite

The command `cargotest--locked--release-ppid-core--all-features` ran on a `git` archive export of each checked commit, with the two evidence generators added as examples:

Commit	Passed	Failed	Ignored
d1f401a	725	0	6
0e96b2b	728	0	6

The extra tests on `0e96b2b` are the three percentile tests of Section 7. The ignored items are one manual kd-tree benchmark, four declared diagnostic tests and one illustrative documentation example that is not compiled. The complete logs are `results/test_suite_d1f401a.log` and `results/test_suite_0e96b2b.log`.

6. Exact test of the one- Λ local bound

6.1 Statement and proof route

Let p and q be probability laws on the same finite alphabet with $\text{supp}(q) \subseteq \text{supp}(p)$. Let $p_{\min} = \min_{z \in \text{supp}(p)} p(z)$, $\delta = \|q - p\|_1 < 2p_{\min}$ and $\eta = \delta/2$. The concentration document, Section 5.3, Equation (11), states that for every supported z , every antichain α and every component $u \in \{+, -, \text{sx}\}$,

$$|i_{\alpha}^u(z; q) - i_{\alpha}^u(z; p)| \leq \Lambda, \quad \Lambda = \log \frac{p_{\min}}{p_{\min} - \eta}.$$

The proof uses the path $p_s = (1-s)p + sq$ for $0 \leq s \leq 1$. Every supported cell of p_s has mass at least $\mu_s = p_{\min} - s\eta > 0$. Put $\Delta = q - p$. Because both laws have total mass one, $\sum_z \Delta(z) = 0$. For any real function g on the support, subtracting the midpoint of the range of g then gives

$$\left| \sum_z \Delta(z) g(z) \right| \leq \eta \left(\max_z g(z) - \min_z g(z) \right).$$

The concentration document tabulates the cell gradients of each cumulative term. For each component, the largest difference between two gradient values is at most $1/p_s(\mathfrak{t} \cap \alpha_{\alpha}) \leq 1/\mu_s$. Integrating along the path gives

$$|i_{\alpha}^u(z; q) - i_{\alpha}^u(z; p)| \leq \int_0^1 \frac{\eta}{p_{\min} - s\eta} ds = \log \frac{p_{\min}}{p_{\min} - \eta} = \Lambda.$$

The net component contains three event probabilities, yet one Λ bounds it. This is the strongest claim in that section, so it received a direct test. Note that the proof needs $\sum_z \Delta(z) = 0$.





6.2 Exact-rational method and results

The script `python/check_one_lambda.py` draws 4000 random laws with seed 1. It uses two or three sources, alphabets of two or three states and random supports. Every law is a vector of integers over one common denominator. The script moves mass η from one to three donor cells to a disjoint set of one to three receiver cells. Hence q is again a probability law, its support equals the support of p , and $\delta = 2\eta$ exactly. One quarter of the trials use a small η , down to $10^{-12}p_{\min}$.

For every antichain and every supported realization, the script computes the three changes $|\Delta i_\alpha^u|$ and the ratio $|\Delta i_\alpha^u|/\Lambda$. It evaluates every ratio from the exact integers with 80-digit decimal logarithms. For the smallest tested Λ , about 10^{-12} , the rounding error of a ratio is below 10^{-66} . The script fails if any ratio exceeds $1 + 10^{-60}$.

Quantity	Result
Trials used	3999 of 4000
Realization–antichain evaluations	634,112
Smallest tested $\eta/p_{\min}$	1.0×10^{-12}
Largest informative ratio	exactly 1 (bound attained)
Largest misinformative ratio	exactly 1 (bound attained)
Largest net ratio	0.9999996544
Violations	none

The informative ratio equals one in 39 evaluations. In each, the source event holds exactly one supported cell, which has mass $p_{\min}$ and loses all of η : $p(\mathbf{a}) = p_{\min}$ and $q(\mathbf{a}) = p_{\min} - \eta$. The misinformative ratio equals one in 51 evaluations. These need the same condition for the intersection event $\mathbf{t} \cap \mathbf{a}$, and also $q(\mathbf{t}) = p(\mathbf{t})$, because the receivers have the same target value. So the bound is sharp for these two components. No evaluation attains it for the net component. The full output is `results/check_one_lambda.txt`.

6.3 Why the retired binary64 test reported ratios above one

The first version of this test, retained as `python/archive/check_one_lambda_binary64_v1.py` with its output, built q in binary64 arithmetic and allowed donor and receiver cells to overlap. It reported a largest ratio of 1.000000000000407 for $\delta \geq 10^{-7}$ and a ratio of 4 in one trial with $\delta < 10^{-7}$. An earlier draft of this report explained the second value as rounding noise in the logarithms. That explanation was wrong.

The diagnostic `python/archive/diagnose_one_lambda_binary64_v1.py` replays the retired test and converts its binary64 arrays to exact rationals. Six trials had $0 < \delta < 10^{-7}$. The third column gives the mass of each cell that changed, divided by $p_{\min}$:

Trial	Donor and receiver cells	Changed-cell mass/ $p_{\min}$	$(\sum q - \sum p)/\delta$	Exact ratio	Binary64 ratio
197	the same one cell	2.55	−1	0.78	0.75
660	the same one cell	1.00	+1	1.77	2.00
2098	the same one cell	1.00	−1	2.00	4.00
2264	the same two cells	5.75 and 35.06	-1.1×10^{-11}	0.17	0.17
2980	the same one cell	1.00	−1	2.00	0.00
3958	the same one cell	1.00	−1	2.00	0.00

In five trials the only donor was also the only receiver. The intended moves cancelled, and $q - p$ is a rounding residue in one cell. So $\sum q - \sum p = \pm\delta$, and q is not a probability law. The premise $\sum_z \Delta(z) = 0$ of Section 6.1 fails. Without it, adding or removing mass δ in one cell of mass m changes a cumulative term by at most about δ/m , while $\Lambda \approx \delta/(2p_{\min})$. The ratio is therefore at most about $2p_{\min}/m$. It lies between 1.77 and 2.00 when the changed cell is the $p_{\min}$ cell, and it is $0.78 \approx 2/2.55$ in trial 197. In trial 2264 the donor and receiver sets were the same two cells, and





the moves nearly cancelled. The net transfer of 3.4×10^{-8} from one cell to the other left q a law to within 7.6×10^{-19} , and the trial obeys the bound. The binary64 ratios are unreliable in this regime: the binary64 value of Λ was 2.2×10^{-16} in the five one-cell trials, although the exact values lie between 6.2×10^{-17} and 2.0×10^{-16} .

The excess 4×10^{-13} in the largest ratio for $\delta \geq 10^{-7}$ has a related cause. In that trial (number 1009), the binary64 construction of q removed 2.4×10^{-17} more mass than it added. Measured against the mass actually removed, the exact ratio is 1 in all 30 printed decimal places. The exact test of Section 6.2 removes both problems. The retired test, its output and the diagnostic output remain as negative evidence.

7. Percentile indices: defect and fix

7.1 Definition and the exact identity

In this section, α is the two-sided tail mass of the resampling summaries, as in the code. It is not an antichain. The summaries sort B replicate values $v_{(0)} \leq \dots \leq v_{(B-1)}$ and report a lower and an upper value for a tail mass $\alpha \in (0, 1)$. Before the fix, the indices were

$$\ell = \lfloor \frac{\alpha}{2} B \rfloor, \quad h = \min \{ \lceil (1 - \frac{\alpha}{2}) B \rceil - 1, B - 1 \}.$$

Claim. In exact arithmetic, $B - 1 - h = \ell$, so both tails drop ℓ values, and $\ell \leq h$.

Proof. Put $x = \alpha B/2$, so $0 < x < B/2$. For every real x and integer B , $\lceil B - x \rceil = B + \lceil -x \rceil = B - \lfloor x \rfloor$. Hence $\lceil (1 - \alpha/2)B \rceil - 1 = B - 1 - \lfloor x \rfloor \leq B - 1$, so the minimum is inactive and $B - 1 - h = \lfloor x \rfloor = \ell$. Also $2\ell \leq 2x < B$, so $2\ell \leq B - 1$, which is $\ell \leq h$. The last step needs $\alpha < 1$. $\square$

7.2 The binary64 defect

The code evaluated ℓ and h with two separate binary64 expressions. The two roundings can disagree. For $\alpha = 0.29$ and $B = 200$, the computed product $\text{fl}(0.145 \cdot 200)$ is 28.999999999999996, so $\ell = 28$. The upper expression rounds to the integer 171, so $h = 170$ and $B - 1 - h = 29$. The summary therefore dropped 28 values below and 29 above.

The script `python/check_percentile_index.py` evaluates both forms exactly as binary64 arithmetic does:

- For the 12 values of α in an earlier probe, from 0.001 to 0.5, and every B from 2 to 20000, the old form never dropped unequal tails. Its lower index agreed with the exact decimal floor $\lfloor \alpha B/2 \rfloor$ in every case. It differed from the exact floor for the binary64 value of α in 1000 cases, all with $\alpha = 0.3$ and B a multiple of 20.
- For all 99 two-decimal values of α and every B from 1 to 20000, the old form dropped unequal tails for 22 values of α , in 2898 pairs (α, B) . The smallest B affected was 25, for $\alpha = 0.88$.

The earlier draft of this report tested only the first grid and concluded that the tails were always equal. That conclusion did not generalize.





7.3 The fix

One helper, `equal_tail_percentile_indices` in `bootstrap.rs`, now computes

$$t = \min \left\{ \left\lfloor \text{fl} \left(\text{fl} \left(\frac{\alpha}{2} \right) B \right) \right\rfloor, \left\lfloor \frac{B-1}{2} \right\rfloor \right\}, \quad \ell = t, \quad h = B - 1 - t.$$

The live routines `block_bootstrap`, `block_bootstrap_paired`, `bootstrap_rows_stats` and `bootstrap_quantized_sxpid2`, with their budget and cancellation variants, reach it through two call sites. The routine `bootstrap_pid3` is compiled out; its block uses the same helper. By construction, both tails drop t values and $\ell \leq h$. The clamp is inactive in exact arithmetic, by the claim above, and it was not active in any tested pair. The lower index equals the old lower index in every tested pair, so lower percentiles do not change there. An upper percentile changes only where the old tails were unequal. Apart from the new tests, every repository call site and test uses $\alpha = 0.05$ or $\alpha = 0.1$, for which the two forms agree for every $B \leq 20000$.

Three new tests cover the fix. The first checks the helper for every two-decimal α with $B \leq 2000$ and for five fixed pairs, including $\alpha = 0.29$ with $B = 200$. The second runs `block_bootstrap`, and the third runs `bootstrap_rows_stats`, each with $\alpha = 0.29$ and $B = 200$. Both check that the reported values are the sorted replicates with indices 28 and 171. Each test fails when its call site uses the old form.

7.4 What the fix does not change

The tail count is the floor of a rounded product, so it does not always equal either exact reference. It is one less than the decimal floor $\lfloor \alpha B / 2 \rfloor$ in 450 of the 1,980,000 two-decimal pairs tested: $\alpha = 0.29$ with $B = 200$ gives $t = 28$, not 29. It is one more than the exact floor for the binary64 value of α in 13,950 pairs. For example, the binary64 value of 0.03 lies slightly below 0.03, so for $B = 200$ its exact floor is 2. The rounded product is exactly 3.0, so $t = 3$, which equals the decimal floor. In every tested pair, t is the decimal floor or one less, and it is the binary64-exact floor or one more. The documentation of every affected field states the index rule. The raw percentiles still have no general coverage guarantee.

8. Re-derived results

Each item below was re-derived step by step. No gap was found.

8.1 Why Levina–Bickel needs $k - 2$ and $k \geq 3$

Assumption. Near a query point, the sample behaves like a homogeneous Poisson process with intensity $\lambda > 0$ in an m -dimensional space. This is Levina and Bickel's local approximation.

Let $T_1 < \dots < T_k$ be the distances to the k nearest neighbours, and let c be the volume of the unit ball. The quantities $U_j = c\lambda T_j^m$ are the first k arrival times of a unit-rate Poisson process on $(0, \infty)$. Given U_k , the values $U_1, \dots, U_{k-1}$ are the order statistics of $k - 1$ independent uniform variables on $(0, U_k)$. Hence $V_j = U_j / U_k$ are, as an unordered set, independent uniform variables on $(0, 1)$, and

$$\log \frac{T_k}{T_j} = \frac{1}{m} \log \frac{U_k}{U_j} = \frac{1}{m} (-\log V_j).$$

Each $-\log V_j$ is exponential with mean 1. Therefore $S = \sum_{j=1}^{k-1} \log(T_k / T_j)$ has the gamma law with shape $k - 1$ and rate m . For $k \geq 3$,





$$\mathbb{E}\left[\frac{1}{S}\right] = \int_0^\infty \frac{1}{s} \frac{m^{k-1} s^{k-2} e^{-ms}}{\Gamma(k-1)} ds = \frac{m^{k-1} \Gamma(k-2)}{\Gamma(k-1) m^{k-2}} = \frac{m}{k-2}.$$

So $(k-1)/S$ has mean $m(k-1)/(k-2)$, which is 12.5% high at $k=10$, and $(k-2)/S$ has mean m . For $k=2$, S is exponential and $\mathbb{E}[1/S]$ diverges at $s=0$. This explains the requirement $k \geq 3$. The mean of the local estimates over sample points is unbiased under the same approximation by linearity of expectation, although the local estimates are dependent.

Levina and Bickel state the normalization in Section 3, below their Equation (8): “One could divide by $k-2$ rather than $k-1$ to make the estimator asymptotically unbiased, as we show below.” Their Equation (9) averages the local estimates arithmetically. They also average over a range of k ; pid-rs uses one k . Reviewer observation: their Section 3.1 writes $m^{-1} \log(T_k/T_j)$ for the standard exponential variables. The derivation above gives $m \log(T_k/T_j)$. Their conclusion $\mathbb{E} U^{-1} = 1/(k-2)$ holds with $U = mS$. This observation has not been confirmed by the authors.

8.2 Simultaneous L^1 radius for the empirical law

Assumptions. The rows are independent with common law P on K cells. The failure probability satisfies $0 < \alpha < 1$; in this subsection α is not an antichain.

For fixed n and cell z , $\hat{P}_n(z)$ is the mean of n independent Bernoulli variables with mean $P(z)$. Hoeffding's inequality gives $\Pr(|\hat{P}_n(z) - P(z)| \geq t) \leq 2e^{-2nt^2}$. Assign the failure budget $\alpha_{n,z} = 6\alpha/(\pi^2 n^2 K)$; these budgets sum to α because $\sum_{n \geq 1} n^{-2} = \pi^2/6$. Solving $2e^{-2nt^2} = \alpha_{n,z}$ gives

$$t_n = \sqrt{\frac{\log(\pi^2 n^2 K / (3\alpha))}{2n}}.$$

A union bound over all n and z gives, with probability at least $1 - \alpha$, that $|\hat{P}_n(z) - P(z)| < t_n$ for all n and z at once. Summing over the K cells gives the repository's radius $\|\hat{P}_n - P\|_1 \leq B_n = K t_n$.

8.3 Dependency-colored concentration

Assumptions. Each row has a predeclared color. Rows of the same color are mutually independent. Rows of different colors may be dependent. All rows have the same law P . Color a has n_a of the first n rows. Only colors with $n_a > 0$ enter below.

Fix a cell set A and let $Y = \sum_i \mathbf{1}\{Z_i \in A\}$ with Y_a the part from color a . For positive weights w_a with $\sum_a w_a = 1$, Hölder's inequality with exponents $1/w_a$ gives

$$\mathbb{E} e^{s(Y - \mathbb{E}Y)} \leq \prod_a (\mathbb{E} e^{(s/w_a)(Y_a - \mathbb{E}Y_a)})^{w_a} \leq \exp\left(\frac{s^2}{8} \sum_a \frac{n_a}{w_a}\right).$$

This is the fractional-coloring argument of Janson (2004). The second step is Hoeffding's lemma for the n_a independent indicators of color a . By the Cauchy–Schwarz inequality, $\sum_a n_a/w_a \geq (\sum_a \sqrt{n_a})^2$ when $\sum_a w_a = 1$, with equality at $w_a \propto \sqrt{n_a}$. The smallest value is therefore $V_n = (\sum_a \sqrt{n_a})^2$. The Chernoff bound with $s = 4t/V_n$ then gives $\Pr(Y - \mathbb{E}Y \geq t) \leq e^{-2t^2/V_n}$. Put $t = n\varepsilon/2$. Because $\|\hat{P}_n - P\|_1 = 2 \max_A (\hat{P}_n(A) - P(A))$, a union bound over the $2^K - 2$ nontrivial sets A gives the repository's bound

$$\Pr(\|\hat{P}_n - P\|_1 \geq \varepsilon) \leq (2^K - 2) e^{-n^2 \varepsilon^2 / (2V_n)}.$$

With one color, $V_n = n$ and this is the bound of Weissman et al. (2003).





8.4 Support-change continuity

The support-change continuity document compares two laws p and q whose supports may differ. Its Sections 1–3 define the objects used here:

- The overlap $r_x = \min\{p_x, q_x\}$ and the residuals $a = p - r$ and $b = q - r$, with $\sum_x a_x = \sum_x b_x = \eta$ (the total-variation distance) and $R = \sum_x r_x = 1 - \eta$.
- The residual entropy $E(d) = -\sum_{x:d_x>0} d_x \log d_x$ of a nonnegative vector d .
- A neighborhood N_x of each cell x , which contains x . For a vector v , write $v(N_x) = \sum_{y \in N_x} v_y$. The functional is $G_N(p) = -\sum_{x:p_x>0} p_x \log p(N_x)$.
- The overlap load $T_N(r, d) = \sum_{x:r_x>0} r_x d(N_x)/r(N_x)$.
- For shared exclusions, N_x is the union of J equivalence classes, one for each collection of the antichain, and $g_J(\eta) = (1 - \eta) \log(1 + J\eta/(1 - \eta))$ for $0 < \eta < 1$.

The key steps were re-derived:

1. The exact split $G_N(p) - G_N(q) = C_N + U_{p,a} - U_{q,b}$, where C_N collects the overlap cells and $U_{p,a} = -\sum_{x:a_x>0} a_x \log p(N_x)$ collects the residual cells.
2. The residual bound $0 \leq U_{p,a} \leq E(a)$. It uses $a_x \leq p(N_x) \leq 1$, which holds because $x \in N_x$.
3. Jensen's inequality for the concave logarithm with weights r_x/R , which bounds the overlap term by $R \log(1 + T_N(r, d)/R)$.
4. The load bound $T_N(r, d) \leq J\eta$. It bounds $d(N_x)$ by the sum over the J classes and $r(N_x)$ from below by one class, and then sums each class once.
5. The pointwise sign facts $0 \leq \pi_\alpha^\pm(z) \leq -\log p_z$ on the full lattice.

The equality witness was recomputed. It uses J sources, a constant target, and the masses $r_{\ell_j} = (1 - \eta)/J$, $a_z = \eta$ and $b_c = \eta$, with neighborhoods $N_{\ell_j} = \{\ell_j, c\}$, $N_z = \{z\}$ and $N_c = \{c, \ell_1, \dots, \ell_J\}$. Then $G_N(p) = -\eta \log \eta - (1 - \eta) \log \frac{1-\eta}{J}$ and $G_N(q) = -(1 - \eta) \log \left(\frac{1-\eta}{J} + \eta \right)$, so

$$G_N(p) - G_N(q) = -\eta \log \eta + (1 - \eta) \log \left(1 + \frac{J\eta}{1 - \eta} \right) = E(a) + g_J(\eta).$$

The monotone envelopes of Section 8 of that document were also checked. For $0 < \eta \leq \varepsilon \leq 1$ and $A \geq 1$,

$$\eta \log \frac{A}{\eta} = \eta \log \frac{A}{\varepsilon} + \eta \log \frac{\varepsilon}{\eta} \leq \varepsilon \log \frac{A}{\varepsilon} + \frac{\varepsilon}{e},$$

because $\log(A/\varepsilon) \geq 0$ and $x \log(1/x) \leq 1/e$ for $0 < x \leq 1$, applied with $x = \eta/\varepsilon$. This envelope is slightly sharper than the one used in that document, and both are valid.

8.5 Other checked items

- **Exact binary64 summation.** Every finite binary64 value is an integer multiple of 2^{-1074} , with a significand below 2^{53} at binary exponent at most 2045 in those units, so its magnitude is below 2^{2098} units. The accumulator keeps two unsigned magnitudes, one for positive and one for negative terms, each with $\lceil (2098 + 64)/64 \rceil = 34$ limbs of 64 bits. The extra 64 bits cover at most `usize::MAX` terms, which is the number of terms the code allows. So each magnitude holds its exact sum. The final rounding selects the top 53 bits and applies round-half-to-even with a sticky bit. The subnormal and overflow branches follow the binary64 encoding.
- **Gaussian closed forms in exp0.** Let S_1, S_2 and Z be independent standard normal variables and let $T = aS_1 + bS_2 + cZ$ with $c \neq 0$. Then $\text{Var}(T) = a^2 + b^2 + c^2$, $\text{Var}(T \mid S_1) = b^2 + c^2$ and $\text{Var}(T \mid S_1, S_2) = c^2$. For jointly Gaussian variables, the mutual information is half the





logarithm of the ratio of the unconditional to the conditional variance (Cover and Thomas, 2006, Chapter 8). This gives the three checked values.

- **Co-information.** $I(S_1; T) + I(S_2; T) - I(S_1, S_2; T) = \text{Red} - \text{Syn}$ for every PID with these atoms. The triplet sum equals Bell's four-variable co-information, and three-bit parity gives $+\log 2$.
- **False-discovery-rate adjustment.** The step-up values $q_{(i)} = \min_{j \geq i} \min\{1, p_{(j)} m c(m)/j\}$ use $c(m) = 1$ for Benjamini–Hochberg and $c(m) = \sum_{i=1}^m 1/i$ for Benjamini–Yekutieli. Tied p -values receive equal q -values.
- **Permutation tail fractions.** The add-one fraction $(1 + b)/(1 + B)$ counts ties as extreme. It is a Monte Carlo p -value under full-row or whole-block exchangeability. For circular shifts the repository correctly labels it a surrogate score, because the admissible shifts exclude the identity and do not form a group.

9. Correction: intrinsic-dimension provenance

Previous text. The code documentation and the method catalog described the $1/(k - 2)$ normalization as the “MacKay–Ghahramani correction”. The catalog linked MacKay and Ghahramani (2005) as the implementation basis for a “k minus 2 finite-sample correction”.

Primary sources. Levina and Bickel (2004), Section 3, below their Equation (8), state the normalization; Section 8.1 quotes the sentence. The MacKay–Ghahramani note recommends a different combination rule: “Rather than averaging the estimators, we think you should average their inverses.” `pid-rs` uses the arithmetic mean of the $(k - 2)$ -normalized local estimates. It follows Levina and Bickel and does not use the MacKay–Ghahramani combination rule. The retrieved bytes were:

Retrieved source	Bytes	SHA-256
Levina–Bickel conference PDF	135,780	5449eb783ff2ced1929979f46b91718bf79f81052b7c4f1f78946cddf5e83603
MacKay–Ghahramani note page	7,426	9c633a8cab18fbc418cbe33449996dea4442d3aa2251d700611e2fa1aa370eb2

The PDF came from the [NeurIPS proceedings](#) on 25 September 2026 and again, with the same bytes, on 26 September 2026. The note page came from <https://www.inference.org.uk/mackay/dimension/> on 25 September 2026. The source bytes are not retained because they are third-party copyrighted material; the hashes let a reader confirm a retrieved copy.

Change. Commit `0e96b2b` corrected the `geometry.rs` documentation, one test comment, the method-catalog locator and summary, and the generated method views. The MacKay–Ghahramani link is kept as a caveat about the alternative combination rule. The estimator code and every numerical output are unchanged. Two publication input profiles received dependency-only successors because they pin the regenerated `METHODS.md`: `recorded-office v6` and `finite-MGW v9`. Exact rebuilds under both profiles reproduced the referenced PDFs byte for byte; `results/publication-profile-rebuilds.txt` retains the commands, exit statuses and digests.





10. Routes considered

Each route lists its assumption, the evidence it would need, the decision, and what would change that decision.

1. **No action; rely on the existing tests.** *Assumption:* the existing oracle and property tests cover the risk. *Evidence needed:* none. *Decision:* rejected. The existing exact oracle covers fixed tables; randomized systems, the $I_{\min}$ lattice and the continuous paths had no second implementation. *Would change if:* a maintained external implementation were compared in CI.
2. **Code reading only.** *Assumption:* careful reading finds arithmetic slips. *Evidence needed:* a line review record. *Decision:* kept as a complement. Reading alone did not find the percentile defect; an exact analysis did. *Would change if:* a named human line review were recorded.
3. **Second from-definition implementation.** *Assumption:* different algorithms from the same definitions expose implementation errors. *Evidence needed:* agreement to rounding on seeded systems. *Decision:* **selected**. *Failure condition:* a shared misreading of a definition passes both implementations. *Would change if:* an independent team wrote the second implementation.
4. **Exact rational oracle for every quantity.** *Assumption:* exact arithmetic removes rounding questions. *Evidence needed:* rational event probabilities and high-precision logarithms. *Decision:* used only for the one- Λ bound. The existing Decimal oracle already covers two-, three- and four-source tables for the categorical atoms. *Would change if:* a rounding discrepancy above 10^{-12} appeared.
5. **Comparison with an external SxPID package.** *Assumption:* an external package implements the same definitions. *Evidence needed:* a pinned external version and matching conventions. *Decision:* deferred. *Would change if:* such a package were pinned in the repository.
6. **Formal proof of the lattice and atoms.** *Assumption:* a formal statement matches the Rust code. *Evidence needed:* a refinement proof. *Decision:* not selected; it would prove definitions, not the Rust code. *Would change if:* a refinement route to the Rust code existed.
7. **Property tests inside the Rust suite.** *Assumption:* invariants catch errors. *Evidence needed:* invariants that do not share the implementation. *Decision:* rejected as the main route because of shared language and author. The percentile fix adds three such tests.
8. **Mutation testing of the Rust code.** *Assumption:* surviving mutants show weak tests. *Evidence needed:* a mutation run. *Decision:* deferred; it measures test strength, not correctness. The three percentile tests were checked against the old form, which acts as one mutant.
9. **Brute force versus kd-tree only.** *Assumption:* two search paths suffice. *Decision:* already in the repository; covered again through the continuous checks and Lemma 2.
10. **Simplest non-PID route: mutual information and entropy only.** *Assumption:* the MI terms determine the atoms. *Decision:* insufficient, because redundancy is not a function of the MI terms. Included as the exact joint-MI sub-check.
11. **Re-derivation of every repository theorem.** *Assumption:* each theorem can be re-derived in one session. *Decision:* partially done in Section 8. *Would change if:* a theorem-by-theorem review plan were adopted.





11. Rust implementation and computational cost

Estimands and APIs. The checks exercise these public functions:

- Default features, empirical plug-in laws: `discrete_sxpid2`, `discrete_sxpid3` and `discrete_sxpid_n` in `pid_core::stable::categorical`; `imin_pid2` and `imin_pid3` in `pid_core::stable::imin`.
- Feature `experimental-continuous`: `ksg_mi`, `ksg_local_mi_terms`, `ksg_mi_concat_xy` and `isx_redundancy` in `pid_core::experimental::continuous::raw_scalars`; `pid2_isx` and `incomplete_pid3_diagnostic` in `pid_core::experimental::continuous`.
- Feature `research-mixed-dimension-pid3`: the research function `pid3_isx` in `pid_core::experimental::mixed_dimension_pid3`.
- Feature `experimental-pipelines`: `block_bootstrap`, `block_bootstrap_paired`, `bootstrap_p_rows_stats` and `bootstrap_quantized_sxpid2`, which use the fixed percentile indices.

All continuous inputs declare the absolute-continuity support contract explicitly.

New code. Commit 0e96b2b adds the crate-internal helper `equal_tail_percentile_indices` and three tests. It adds no public API. The two evidence generators are copied into a temporary export only; they are not part of the published crate.

Resource limits and failures. The generators call the entry points that apply each function's default resource budget. The discrete generator stops with a panic, and therefore a nonzero exit status, if `pid-rs` returns any error; `run.sh` then stops. The continuous generator does the same for every call except the research full PID3, whose error text it records in the output instead. The continuous checker fails if that lattice is absent. No cancellation is used because every call is small.

Cost of the checks. The Python categorical check iterates only over the observed realizations R of each system, not over the full product alphabet. For each realization it evaluates the event masses of all $|\mathcal{A}_m|$ antichains by scanning the R observed realizations, which costs $O(R^2|\mathcal{A}_m|)$ membership tests per system. It then performs two dense $|\mathcal{A}_m| \times |\mathcal{A}_m|$ solves per realization, one for each component. For four sources, $|\mathcal{A}_4| = 166$, so each matrix has 27,556 entries. The continuous check stores dense $N \times N$ distance matrices and sorts one row per query, which costs $O(N^2)$ memory and $O(N^2 \log N)$ time per estimator. All checks are offline batch computations on stored rows. The measured times are in `results/timing_0e96b2b.txt`: the complete run on 0e96b2b, with a cold release build and the test suite, took 160 s of wall time on an Apple M4 Max with macOS 26.5.1, rustc 1.96.0, Python 3.14.6 and NumPy 2.4.6. These are measurements of the checks, not benchmarks of `pid-rs`.

12. Review record

In the first review round, two model-review sessions reported on a draft: one for mathematics and statistics, one for provenance and consistency. The two reports found five main problems:

- the percentile defect;
- the wrong explanation of the retired binary64 test;
- the wrong section number of the Levina–Bickel locator;
- an existing four-source oracle test that the draft had missed; and
- the missing independence record.





A second session reviewed the revised draft for mathematics and code. The author session performed the language, layout and consistency lenses and the formal-package audit of Section 14. `REVIEW.md` records every lens, finding and disposition. The repository gates that apply to this change are listed there with their outcomes.

13. Reproduction

From a checkout that contains this report, run:

```
audit/evidence/cross-implementation-reverification-2026-09-26/run.sh <commit>
↪ <fresh-work-dir> --with-test-suite
```

The work directory must not exist and must lie outside the checkout. The script exports the commit with `git archive` and copies the two generators into that export. It runs the generators with a private Cargo target directory. It then runs the four Python checks, the checker self-test, the SMT obligation check and its normal/optimized protocol controls (Section 14), and the archived binary64 diagnostic. The SMT check needs Z3 4.16.0 on the path. Finally, it compares each output with the retained copy and fails on any difference. After the outputs were retained, a second run on each commit matched all eight retained outputs of that time byte for byte. After the SMT check was added, a run on commit `e1e0b81` matched all nine; `results/reproduction-confirmation.txt` records these runs. `MANIFEST.json` lists every retained file with its size and SHA-256.

14. Semantic audit of the formal packages

A formal proof shows that a Lean theorem or an SMT obligation follows from its premises. It does not show that the statement means what the prose says. This section records a separate audit of that correspondence for the 14 Lean directories under `audit/formal/lean*` and the two SMT packages `audit/formal/z3` and `audit/formal/z3-ksg-harmonic`, on commit `e1e0b81`. The author session performed the audit. Model-review sessions contributed the checks that the table attributes to them. The audit did not re-run the Lean replays; hosted CI runs them.

14.1 Questions asked of every package

1. Does the formal statement say what the prose claims: the same objects, quantifiers, premises, logarithm base and conventions?
2. Do the definitions match the defining source? For MGW, a source event is a disjunction over the collections of an antichain, each collection contributing the conjunction of its sources. The order is $\beta \preceq \alpha$ when every set in α contains a set in β . The atoms are the Möbius inverse of the cumulative terms.
3. Can the premises be satisfied? A theorem with contradictory premises is true but empty.
4. Is the trust base clean? Accepted sources must not contain `sorry`, an axiom declaration, `native_decide`, `implemented_by`, `extern` or `unsafe`, and axiom audits may list only `propext`, `Classical.choice` and `Quot.sound`.





14.2 Mechanical scan

No accepted Lean source contains any of the tokens above. The only **sorry** and axiom declarations are in **audit/formal/lean-mgw-fixed-world/negative**. Its nine mutation controls change one theorem each: a weakened statement, a redefined target, a renamed or re-kinded declaration, a wrapped statement, an injected or imported axiom, and a **sorry**. The judge must reject each of them. The pinned Lean 4.33.0 contains the fix for Lean issue 14576, a kernel soundness bug fixed on 28 July 2026, and the repository retains its upstream regression test.

14.3 Results by package

Package	What was checked	Result
lean (finite convergence)	The MGW event and target restriction in the semantic contract; all 16 entries of the two-source Möbius and zeta matrices; the fixture counts, component arguments and weighted products $8/3$, $2/3$ and 1 , recomputed by hand; the counts of 339 declarations and 246 theorems; the refined diamond bound $1/x_a - 1/S$ and its attaining pair; the $\lfloor K^2/4 \rfloor$ lemma. The support-change document correctly lists its two-law transfer as proved in prose only	Faithful
lean-finite-logscore	Measure integrals of a PMF; supported entropy and KL in nats; the Gibbs premise; the log-score identity; the conditional continuation (13 exports, 37 helpers, recorded hash)	Faithful
lean-exact-log-product	Scope statement; the five-term product equals one	Faithful
lean-foundational-sxpid	Scope statement: a generic factorization firewall	Faithful
lean-citation-edge	Scope statement: a finite exact-sequence countermodel with no PID claim	Faithful
lean-kernel-regression	Lean issue 14576 and its upstream fix	Faithful
lean-mgw-fixed-world	Synergy $\log(4/3)$ for both added sources and conditional MI 0 and $\log 2$, recomputed by hand and matched with the targets; the negative controls	Faithful
lean-mgw-target-copy	The local identity, its bounds and the averaged bound, derived by hand (below); the replay statement against its record. Its publication record was stale (Section 14.5)	Faithful
lean-sx-dnf-order	Order, DNF events, event inclusion, antisymmetry and injectivity on antichains over pattern-realizing domains; the accepted source bytes and hash	Faithful
lean-sxpid3-informative-invariance	All 16 statements: the informative component depends only on the source marginal; the checker's own axiom audit	Faithful
lean-prefix-mgw-bias	Five accepted targets against the exports; references	Consistent
lean-prefix-mgw-gradient	Eleven targets against the local acceptance record	Consistent
lean-prefix-mgw-mean	Three targets against the acceptance record	Consistent
lean-ksg-harmonic	19 theorems; the digamma premise $\psi(m) = H_{m-1} - \gamma$, which the true digamma satisfies; the exclusive argument $n + 1$; the range rewrite; the sharp bound $ t \leq H_{N-1} - H_{k-1}$; the count ranges, which are explicit premises	Faithful
SMT, nine files	Each complete file is unsat under Z3 4.16.0; each background without its negated goal is sat, so no unsat result is vacuous; the PID3 zeta sums equal the MGW down-sets of all 18 antichains	Faithful

For the three prefix-MGW packages, model-review sessions checked the contract-to-target match and the quoted values and figures; the judges enforce the alias-to-target match.

For the target-copy family, the target is $T = (S_1, B)$. Let a , c and j be the masses of the events $S_1 = s_1$, $S_2 = s_2$ and $S_1 = s_1 \wedge S_2 = s_2$ at an anchor. Because $T = t$ implies $S_1 = s_1$, the four local





MGW terms are $\log(1/a)$, $\log \frac{r}{cp}$, $\log \frac{r}{jp}$ and $\log \frac{1}{a+c-j}$, where r is the mass of $S_2 = s_2 \wedge T = t$ and p the mass of $T = t$. The masses r and p cancel in the synergy:

$$s = \log \frac{ac}{j(a+c-j)}.$$

Since $j \leq \min(a, c)$, $ac - j(a+c-j) = (a-j)(c-j) \geq 0$, so $s \geq 0$. Since $a+c-j \geq \max(a, c)$, $s \leq \log \frac{\min(a,c)}{j}$. Averaging the two upper bounds over the source law gives $\min\{H(S_1 | S_2), H(S_2 | S_1)\}$. These are the Lean targets.

14.4 Why the satisfiable-background test matters

An SMT proof shows that the negated claim, together with the background assertions, has no model. If the background assertions alone had no model, every claim would be proved. Removing the negated goal and finding a model rules this out. `python/check_smt_obligations.py` performs this test for all nine files and the PID3 order check. It fails when a contradictory assertion is added to a background, or when one atom is removed from a PID3 zeta sum.

14.5 Correction

The publication record of the target-copy note named the Markdown and PDF bytes and the input profile from before the author credit of 23 September 2026. Commit `e1e0b81` updates the record to the current bytes and the v5 profile, keeps v4 as the historical profile, and limits its all-page visual review to the v4 bytes. No theorem, value or proof changed.

14.6 Limits

The audit compares statements with prose and re-derives the central identities by hand. It is not a line-by-line review of every proof, and it does not re-run the Lean replays. A statement that matches its prose does not establish estimator calibration, population validity or a Rust refinement; each package states those boundaries itself.

14.7 Follow-up: checker rejection gaps

A subsequent independent-first source review and copied-input tests found two defects in the new SMT checker. It read only the first word from Z3 and ignored the process status and remaining output. Thus `unsat` followed by a solver error still passed. Its lattice comparison extracted atom names but ignored their coefficients. Negating one atom definition and every reconstruction use left the solver identities true and passed the name check, although those reconstruction terms no longer had the claimed coefficient `+1`.

The corrected checker requires a clean process and one exact result, checks the single-query command profile, and parses each reconstruction as the exact positive down-set sum. Its causal controls retain the original failures. The nine original SMT files are unchanged; these findings concern the evidence checker, not a counterexample to those files. The package table contains 14 Lean directories: the 13 `lean-*` directories and `lean` itself. The earlier count of 13 omitted this distinction.

The companion fault-tolerance down-set check also accepted nonfinite JSON atoms and silently truncated unequal row lists. It now validates finite numbers, row shapes, category types and the complete antichain carrier before comparison, and rejects nonfinite arithmetic before updating an error maximum. Its 26 causal controls pass in normal and optimized Python; the historical checker fails seven of them. The retained 60-system output is unchanged. The `correction` record binds these checks; no scientific claim is upgraded.





15. References

- A. J. Bell (2003). The co-information lattice. *Proc. 4th Int. Symp. on Independent Component Analysis and Blind Signal Separation*, 921–926.
- Y. Benjamini and Y. Hochberg (1995). Controlling the false discovery rate: a practical and powerful approach to multiple testing. *J. R. Stat. Soc. B* 57(1), 289–300.
- Y. Benjamini and D. Yekutieli (2001). The control of the false discovery rate in multiple testing under dependency. *Ann. Statist.* 29(4), 1165–1188.
- T. M. Cover and J. A. Thomas (2006). *Elements of Information Theory*, 2nd ed. Wiley.
- D. A. Ehrlich, K. Schick-Poland, A. Makkeh, F. Lanfermann, P. Wollstadt and M. Wibral (2024). Partial information decomposition for continuous variables based on shared exclusions: analytical formulation and estimation. *Phys. Rev. E* 110, 014115. [arXiv:2311.06373v3](https://arxiv.org/abs/2311.06373v3).
- W. Hoeffding (1963). Probability inequalities for sums of bounded random variables. *J. Amer. Statist. Assoc.* 58(301), 13–30.
- S. Janson (2004). Large deviations for sums of partly dependent random variables. *Random Structures Algorithms* 24(3), 234–248.
- A. Kraskov, H. Stögbauer and P. Grassberger (2004). Estimating mutual information. *Phys. Rev. E* 69, 066138.
- E. Levina and P. J. Bickel (2004). Maximum likelihood estimation of intrinsic dimension. *Advances in Neural Information Processing Systems* 17, 777–784.
- D. J. C. MacKay and Z. Ghahramani (2005). Comments on “Maximum likelihood estimation of intrinsic dimension” by E. Levina and P. Bickel. Online note, <https://www.inference.org.uk/mackay/dimension/>.
- A. Makkeh, A. J. Gutknecht and M. Wibral (2021). Introducing a differentiable measure of pointwise shared information. *Phys. Rev. E* 103, 032149. [arXiv:2002.03356v5](https://arxiv.org/abs/2002.03356v5).
- T. Weissman, E. Ordentlich, G. Seroussi, S. Verdú and M. J. Weinberger (2003). Inequalities for the L1 deviation of the empirical distribution. HP Laboratories technical report HPL-2003-97R1.
- P. L. Williams and R. D. Beer (2010). Nonnegative decomposition of multivariate information. [arXiv:1004.2515v1](https://arxiv.org/abs/1004.2515v1).

